

JSON Beginner's Guide

A simple, readable guide to understanding and working with JSON

1. What is JSON?

JSON stands for *JavaScript Object Notation*. Think of it as a clean text format used to store and send data between a server and a website or application.

It is human-readable, simple, and works with almost all programming languages.

2. A Simple JSON Example

Here is what JSON looks like in real life:

```
{
  "name": "Pukar",
  "age": 22,
  "isStudent": true,
  "skills": ["Python", "JavaScript"],
  "address": {
    "city": "Kathmandu",
    "country": "Nepal"
  }
}
```

3. What Types of Data Can You Put in JSON?

JSON only supports 6 simple data types:

- **String:** Text wrapped in double quotes (e.g., "Hello").
- **Number:** Integers or decimals without quotes (e.g., 42 or 3.14).
- **Boolean:** Lowecase true or false.
- **Null:** Represents an empty or missing value (null).
- **Array:** An ordered list enclosed in square brackets [].
- **Object:** A collection of key-value pairs enclosed in curly braces { }.

4. Golden Rules of JSON Syntax

Remember these 3 rules to avoid syntax errors:

- **Always use double quotes** around key names and strings (e.g., "name": "Pukaɾ"). Single quotes will break your code.
- **No trailing commas!** Never leave a comma after the last item in an object or list.
- **Keys must be strings.** You cannot use unquoted words or numbers as keys.

5. How to Get (Index) Values

When you read JSON in programming code, you get values by referencing their key names.

PYTHON EXAMPLE

```
# Suppose 'data' is loaded into Python as a dictionary
data = {
    "user": {"name": "Pukar", "role": "Admin"},
    "tags": ["AI", "Python"]
}

# 1. Get nested values using key names in brackets
print(data["user"]["name"]) # Output: Pukar

# 2. Get items from a list using 0-based index numbers
print(data["tags"][0])      # Output: AI
```

JAVASCRIPT EXAMPLE

```
// In JavaScript, you can use dot notation or bracket notation
const data = {
  user: { name: "Pukar", role: "Admin" }
};

// Dot notation (cleanest)
console.log(data.user.name); // Output: Pukar

// Bracket notation
console.log(data["user"]["role"]); // Output: Admin
```

6. How to Change a Specific Value

To update a value, simply target its key and assign a new value to it.

PYTHON UPDATE WORKFLOW

```
import json

json_string = '{"project": "Load Forecast", "status": "pending"}'

# Step 1: Parse the JSON string into Python
data = json.loads(json_string)

# Step 2: Update the value by targeting its key
data["status"] = "completed"

# Step 3: Convert it back to a JSON string
updated_json = json.dumps(data, indent=2)
print(updated_json)
```

JAVASCRIPT UPDATE WORKFLOW

```
const jsonString = '{"project": "Load Forecast", "status": "pending"}';

// Step 1: Parse JSON text into a JavaScript Object
const data = JSON.parse(jsonString);

// Step 2: Target the key and change its value
data.status = "completed";

// Step 3: Convert back to JSON string
const updatedJson = JSON.stringify(data, null, 2);
console.log(updatedJson);
```